

Computer Science Education for Visually Impaired Students: A Literature Review

A Thesis submitted to the faculty of
San Francisco State University
In partial fulfillment of
the requirements for
the Degree

Master of Arts

In

Education: Special Interest Area

by

Rachel Roslyn Fenichel

San Francisco, California

May 2026

Copyright by
Rachel Roslyn Fenichel
2026

Certification of Approval

I certify that I have read Computer Science Education for Visually Impaired Students: A Literature Review by Rachel Roslyn Fenichel, and that in my opinion this work meets the criteria for approving a thesis submitted in partial fulfillment of the requirement for the degree Master of Arts in Education: Special Interest Area at San Francisco State University.

Doris Flowers, Ph.D.

Professor

Thesis Committee Chair

Adam Graves, Ph.D.

Associate Professor

Abstract

Block-based programming environments are commonly used in early computer science education, but present accessibility barriers to students with visual impairments. Researchers have explored the accessibility of block-based programming environments from the perspective of human-computer interaction and computer science education. The barriers can be described as age-related, social, and technical factors. Adding keyboard navigation to the Blockly library can address many of these barriers.

Acknowledgements

Thank you to Nathan for supporting and challenging me. Thank you to Michael for asking good questions and quacking when needed. Thank you to my parents for teaching me how to read and write.

Table of Contents

	2
List of Tables	3
List of Figures	4
Chapter 1	1
Introduction	1
Author Positionality	2
Background	2
Purpose	5
Methods	5
Chapter 2	8
Review of the Literature	8
Barriers to Access	9
Age-related Factors	10
Social Factors	11
Technical Factors	14
Tested Strategies	18
Alternatives to Drag and Drop	18
Tactile interfaces	19
Quorum Blocks	23
Navigation	24
Blockly modifications	27
Guidelines	32
Chapter 3	34
Conclusion	34
Opportunities for Future Work	34
Cursors	35
Addressing Social Barriers	35
Addressing Age-Related Barriers	36
Addressing Technical Barriers	37
Meeting accessibility standards	38
References	39

List of Tables

Table 1. Search terms 5

List of Figures

Figure 1. An example of block-based code.	1
Figure 2. The Scratch programming environment	4
Figure 3. Blocks connected with notches	20
Figure 4. The Quorum Blocks editor	23

Chapter 1

Introduction

Computer science education is an important part of modern K-12 education across the world. In the United States 12 states have computer science graduation requirements and 60% of high schools offer a foundational computer science course (Yongpradit et al., 2025, pp. 2–3). Block-based programming environments (BBPEs) present code as visual blocks instead of punctuation-heavy text. BBPEs are widely used in K-12 computer science education but are generally inaccessible to students with visual impairments due to their heavy use of visual metaphors. In this paper I review the literature on accessibility of block-based programming environments in the K-12 classroom with a focus on visual impairment. I combine the literature on assistive technology and computing education to propose a path forward for the creation of accessible block-based programming environments for use in K-12 educational contexts.

Figure 1. An example of block-based code.

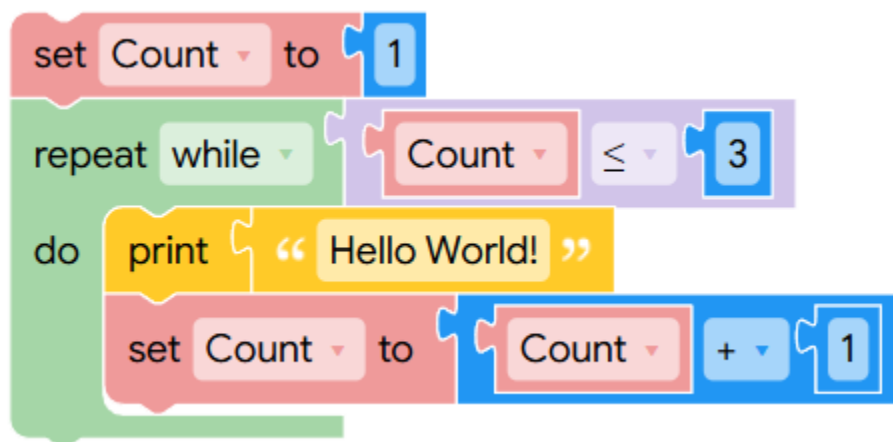


Figure 1. Code to print the text "Hello World" three times represented as colorful blocks that interlock with puzzle pieces

Author Positionality

I am the Director of Blockly at the Raspberry Pi Foundation. Blockly is a software library for creating BBPEs that is “the foundation for some of the largest block-based coding products, such as Scratch and Code.org” (Google for Developers, 2024). I have worked on block programming for 12 years doing both technical and product-level work. My work on Blockly motivated me to pursue my Master’s in Education at San Francisco State University, where I have explored the intersection of computer science education and accessibility. As Director of Blockly I set the technical direction of the team and identify priorities, including the accessibility of products and services. I investigated this topic to understand how best to improve the accessibility of BBPEs through changes to the Blockly library and advocacy within the block-based programming ecosystem.

Background

The Individuals with Disabilities Education Act (IDEA) defines visual impairment as “an impairment in vision that, even with correction, adversely affects a child’s educational performance. The term includes both partial sight and blindness” (Individuals with Disabilities Education Act, 2004). Legal blindness is a more specific term that refers to a visual acuity of 20/200 or less in the better eye or a visual field of 20 degrees or less. According to the American Foundation for the Blind, in 2023 approximately 55,000 legally blind people were involved in education below the college level in the United States (*Statistics on Children*, 2026). They use a variety of assistive technologies such as screen readers, screen magnifiers, and braille output devices. The IDEA requires that “to the maximum extent appropriate, children with disabilities

[...] are educated with children who are not disabled” (Individuals with Disabilities Education Act, 2004). As a result, most visually impaired students in the United States are educated in mainstream classrooms alongside sighted children.

While the IDEA establishes requirements for a free and appropriate public education in the United States, Title II of the Americans with Disabilities Act (ADA) requires that “state and local governments must provide people with disabilities an equal opportunity to benefit from all of their programs, services, and activities” (*Introduction to the Americans with Disabilities Act*, 2026). Together Title II and the IDEA mean that children with visual impairments are owed access to all parts of their education. The 2020 COVID-19 pandemic and resulting shift to remote learning exposed the inaccessibility of many e-learning programs, resulting in an unequal education for students with disabilities. In 2024 the Department of Justice published a rule making clear that web content provided by state and local governments—including school districts—is covered under Title II of the ADA (*Fact Sheet*, 2024). The rule established the Web Content Accessibility Guidelines (WCAG) as the technical standard and mandated compliance with WCAG 2.1, Level AA.

Block-based programming environments represent pieces of code as visual blocks that can be snapped together like puzzle pieces. BBPEs are heavily used in K-12 education and early computer science education because they remove the possibility of syntax errors. BBPEs have been shown to increase learning gains when compared to text-based programming (Weintrop & Wilensky, 2017). Educational BBPEs often integrate an execution environment and application-specific commands designed to introduce students to programming concepts. For example, the Scratch programming environment shown in Figure 2 lets the user program characters that move

and interact. In some cases no equivalent text-based environment exists. Curriculum designers employ BBPEs to incorporate computational thinking into other subject areas, as when a student creates an animated story as part of a book report. As a result inaccessible programming environments impede visually impaired students in their learning in multiple subjects.

Figure 2. The Scratch programming environment

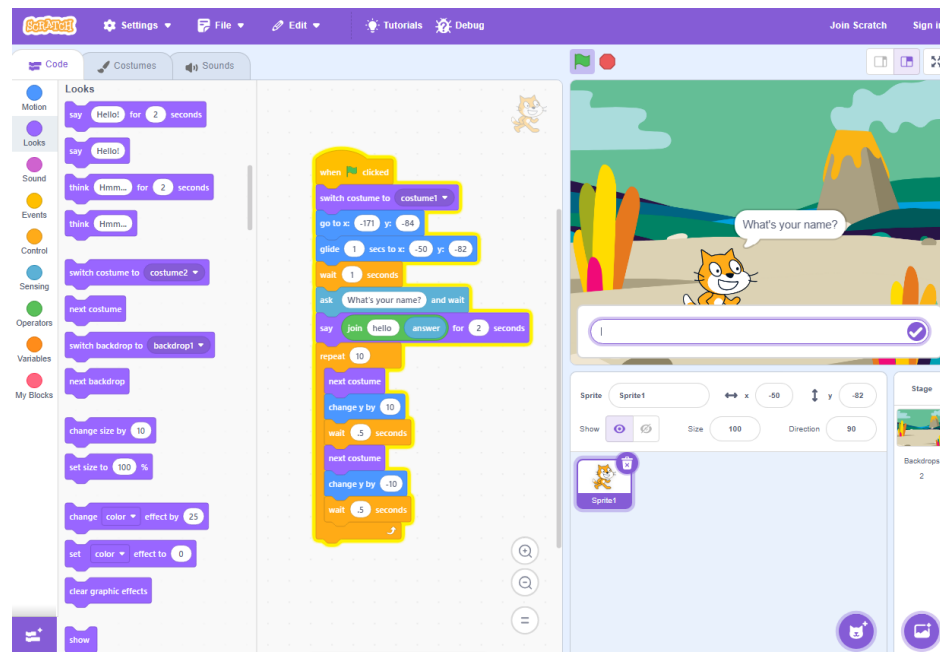


Figure 2. A web page containing a block-based programming interface on the left with a script controlling a cat sprite's movement and text. The stage on the right shows the cat asking the user for their name.

Users program in a BBPE by dragging and dropping blocks to connect in stacks that represent programs. The visual and spatial metaphors used by BBPEs have historically presented an accessibility barrier to students with visual impairments. In light of the changing regulatory environment, entities that develop curriculum and tools for block-based programming in the classroom are struggling to improve the accessibility of their tools and environments. WCAG guidelines and conventions are not designed for the virtual canvas of a block-based programming

environment, making them impossible to apply directly to BBPEs. Failure to meet WCAG exposes schools and districts to civil rights lawsuits. Unless they are made WCAG compliant BBPEs are at risk of being removed from schools for all students. To keep BBPEs in schools it is necessary to define the features of an accessible, WCAG-compliant BBPE.

Purpose

The purpose of this literature review is to examine the existing literature on visual impairment and block-based programming in education, to identify research-backed practices to improve the accessibility of BBPEs for students with visual impairments. To that end, my three research questions are:

- RQ1: What is the state of the literature on BBPEs for students with visual impairments?
- RQ2: What features of BBPEs make them inaccessible for students with visual impairments?
- RQ3: What strategies have been proposed and tested to make BBPEs accessible for students with visual impairments?

Methods

Given the cross-disciplinary nature of my inquiry, I began by identifying journals and conferences that might intersect with my topic. I looked for journals in the fields of human-computer interaction, computer science education, special education, and visual impairment.

Table 1. Search terms

Journal or conference titles	Search terms
Human-Computer Interaction (HCI), Special Interest Group on Accessible Computing (SIGACCESS),	(visual impairment OR blindness OR disability) AND (block-based

Human Factors in Computing Systems (CHI)	programming OR block programming) AND (education OR student OR K-12)
Interaction Design and Children (IDC)	(visual impairment OR blindness OR disability) AND (block-based programming OR block programming)
Exceptional Children, The Journal of Special Education, Remedial and Special Education, Journal of Special Education Technology	(visual impairment OR blindness OR disability) AND (block-based programming OR block programming) AND (computer science OR computational thinking)
Journal of Educational Computing Research, Special Interest Group on Computer Science Education (SIGCSE), Transactions on Computing Education (TOCE)	(visual impairment OR blindness OR disability) AND (block-based programming OR block programming)
Journal of Visual Impairment & Blindness	(block-based programming OR block programming) AND (education OR student OR K-12) AND (computer science OR computational thinking)

Table 1. Journals and conferences searched, grouped by the applied search terms.

For each journal or conference I ran a search using the key search terms that were not implied by the journal's subject area. For instance, the Journal of Visual Impairment & Blindness necessarily includes visual impairment so I searched for "(block-based programming OR block programming) AND (education OR student OR K-12) AND (computer science OR computational thinking)". I also applied a forward snowballing approach by identifying key papers such as Ludi's 2015 position paper and searching for later papers which cited the key papers.

I screened the articles by title and abstract to check for alignment with my inclusion criteria. I included papers that discussed physical programming systems if they also included

discussion of the accessibility of BBPEs. I excluded papers that focused on adults or addressed other forms of disability.

Chapter 2

Review of the Literature

The accessibility of BBPEs for visually impaired students represents the intersection of multiple fields and very few papers address it directly. Instead, researchers have investigated portions of the problem such as how blind programmers normally work. Many of these papers were published in human-computer interaction (HCI) journals. These papers investigated how blind people use computers and assistive devices in a variety of contexts, from the classroom to programming to everyday life.

The field of computer science education research yielded papers on ways to teach programming to blind students. Some of these directly addressed block-based programming, either by identifying barriers or by proposing alternatives. Education papers often surveyed educators to understand what tools they used in their classrooms, while human-computing interaction papers tended to interview adults.

In 2015 Ludi, a key researcher in the field of accessible BBPEs published a position paper titled “Towards making block-based programming accessible for blind users” that defined the problem space and justified research in this area. Since then Ludi and her lab have explored the intersection of visual impairment and programming, including block-based programming. This work began with exploratory studies and progressed to modifying the Blockly library and testing new interfaces with blind programmers. Ludi and her co-authors often publish at the SIGACCESS Conference on Computers and Accessibility. Ludi’s group has explored the impact of visual impairment on the usage of educational BBPEs but their user tests focus on adults.

The AccessCSforAll group is a national research-practitioner partnership run out of the University of Washington which works to improve access to computer science education for students with all types of disabilities. Researchers from this group have published papers on accessible computer science education for blind students. Some of these papers use block-based environments while others use tangible or text programming.

Several parallel bodies of literature are relevant to the accessibility of programming but ultimately beyond the scope of this literature review. One covers the accessibility of text-based programming environments in non-educational contexts (e.g. for professional programmers). Another explores the mental models used by experienced blind programmers. The conclusions of these studies do not generalize to educational BBPEs because novice programmers using BBPEs must simultaneously build mental models of programming concepts and the programming environment. Professional programmers rely on schemas that depend on their past experience and tools and therefore have different needs.

Barriers to Access

The literature shows a wide range of features of BBPEs that represent barriers to access for visually impaired students. These barriers can be organized into three groups: age-related factors, social factors, and technical factors. Age-related factors refer to accessibility barriers that are specific to blind and low vision youth. Social factors are barriers created by the social dynamics of a classroom containing both sighted and visually impaired youth. Technical factors are created by the implementation of BBPEs, such as the accessibility of program inputs and outputs. Taken together these barriers describe the issues that must be addressed by any fixes for the accessibility of educational BBPEs.

Age-related Factors

BBPEs are often used for computer science education in late elementary and middle school, corresponding to ages 8-14. In her 2024 study on BLV youth's digital access needs in school Shaheen interviewed youth users of assistive technology (AT) to understand how they have different needs from adults. She identified two reasons for this distinction: first, that the accessibility needs of youths "differ from adult needs because humans' developmental profiles (e.g. cognitive, social, fine motor) shift as they age" (p. 2); and second, that adults often become blind later in life, and thus develop critical models as sighted youth.

In this study Shaheen (2024) spoke to youth between 13 and 21 years of age. She found that "the majority of the youth (n = 13) began developing their AT literacy in third grade or later, and four of those 13 did not begin the work until middle or high school" (p. 14). Even for students who began using AT in elementary school, "middle school is when learning AT became more of a focus" (p. 14). One student who began learning AT in kindergarten said "'I am very grateful. It's definitely a blessing. A lot of kids are less fortunate'" (p. 14). Furthermore, a large portion of the participants were either "self-taught" or "partially self-taught". This required "the metacognitive awareness to know (1) what they did not know and (2) what kind of support they needed to develop the missing knowledge" (p. 14). That type of metacognitive awareness is often lacking in elementary school students. The youth also felt that "interfaces that they could navigate with the knowledge they already possessed were accessible, and interfaces that required digital navigation techniques they had not yet learned were inaccessible" (p. 18). This means that their definition of accessibility is fluid and changes over time, but points to a decrease in acceptable complexity when working with younger learners.

Mountapmbeme and Ludi (2021) interviewed Teachers of Students with Visual Impairments (TSVIs) to understand the barriers that they saw in their work with visually impaired children in school settings. Most of their participants said that students struggled to learn to code because of a lack of basic computing skills. The teachers reported that “students who had more experience with these skills were better and faster in completing tasks than those who did not” (p. 5) and TSVIs often spent time introducing and teaching basic computing skills in order to access programming lessons. The TSVIs said that their visually impaired students should be introduced to basic computing skills earlier, so that they would be comfortable by the time they encountered programming activities. A similar issue arose with orientation and mobility (O&M) skills because introductory programming environments commonly use spatial concepts.

Baker, Milne, and Ladner (2019) conducted a similar survey in their paper “Understanding the Impact of TVIs on Technology Use and Selection by Children with Visual Impairments”. Their interviewees “highlighted learning typing as one skill that is both especially important and challenging to teach students with visual impairments” (p. 7). However, keyboarding and basic computer skills take time to learn and there is a limited amount of instructional time available before students encounter coding lessons in their mainstream classes. Sighted children are not expected to have strong keyboarding or computing skills before being exposed to beginning programming environments.

Social Factors

Baker, Milne, and Ladner (2019) also discussed “the relationship between the technology the students are using and their disability identity” (p. 7). Some students were reluctant to use

assistive technology because they wanted their disability to remain invisible to their classmates, even when avoiding assistive technology negatively impacted their access to information. Other students avoided using assistive technology when it did not “match their internal disability identity” (p. 7). Students with degenerative conditions were particularly impacted by the mismatch with their internal identity, associating learning advanced AT with their future loss of vision. TSVIs tried to counter these negative perceptions by downplaying disability and connecting technology and skills to common usage patterns for sighted users. The authors argued that mainstream technology features should support all types of visual impairment, including low vision, and recommended that companies improve built-in technology instead of expecting people with low vision to use specialized assistive technology. They also stated that “making accessibility features on technology unobtrusive is especially important for children and teenagers, who might not yet be comfortable with their disability and are still forming their identity” (p. 11).

In a second study Baker, Bennett, and Ladner (2019) interviewed blind programmers to understand their educational experiences in computing classes in college. This was a retrospective study with adults and not specific to block-based programming. Nevertheless, the survey responses provide insights into barriers that are specific to programming education. The students said that faculty support was critical to making their courses accessible and that they chose their courses based on how accommodating faculty members were willing to be. Textbooks and course materials for programming classes were often inaccessible by default due to their use of specialized syntax and formatting. As a result, either the students or the professors had to do extra work to create accessible course materials. Students participating in group

projects often ran into issues with the inaccessibility of tools used during group work, which they found isolating and which often reduced their ability to contribute equally. In turn, this had negative impacts on their socialization with other students.

Social factors also influence how children and students learn to use their assistive technology. Shaheen (2024) refers to the finding that BLV youth in mainstream classrooms may not be taught about AT because their sighted parents or teachers may not be aware of such AT. Youth at specialized schools for the blind were “1.8 times more likely to use AT than their public school peers” (p. 3), in part because of their better access to AT instruction. One participant noted that “when a teacher doesn’t know how to use something they can’t really coach you in how to do it” (p. 15). Students in mainstream schools are therefore at a disadvantage because they are not part of the blind community and able to learn from other blind people who know how to use AT.

Huff et al. (2021) interviewed TSVIs to understand their views of accessible K-12 computing education and found that tools, course materials, curriculum, and teacher training all influenced the perceived accessibility of K-12 computing education. Teachers said that their biggest obstacle was “the lack of accessible learning materials, which then falls on the teachers to produce” (p. 7). Two of the participants believed that “in traditional classrooms with sighted and BVI students [...] the difference between information processing between the group may hurt the BVI student’s confidence, efficacy, and potentially their interest in CS” (p. 6). This study demonstrated the importance of coordinated efforts to improve accessibility. TSVIs need training on computer science to support their students, while computer science teachers need accessible course materials and curriculum.

The papers discussed so far grouped together blind and low vision participants. Szpiro et al. (2016) investigated how people with low vision used computers and phones to complete everyday tasks. Their work was novel because it addressed people with low vision as a separate group from those with no usable vision. People with low vision are sometimes able to “pass” for sighted without using AT or are able to use primarily mainstream technology such as pinch-to-zoom. The study had 11 adult participants, with ages ranging from 20 to 68. Study participants were asked to complete common tasks such as writing grocery lists and selecting from a take-out menu. Researchers observed the participants and asked questions. Participants used a range of AT, including screen magnification, inverted colors, text-to-speech, and phone-as-magnifier. For all but one participant, “discomfort disclosing their disability impacted their use of technology” (p. 7). One participant “found it difficult to communicate at work because she inverted colors on her devices” (p. 6), pointing to the use of computers to facilitate communication in both work and social settings. As with the students, participants varied widely in their knowledge of accessibility tools and some “did not know what was available or how to operate [it]” (p. 7). Although social pressures played a role in their decisions on how and when to use assistive technology, technical factors also played a large role.

Technical Factors

Participants in Szpiro et al. (2016) complained about the inefficiency of using their assistive technology. On-screen magnifiers presented issues because they reduced the user’s field of view and required extensive panning, which led participants to miss items that were on-screen but not in the magnified region. Sometimes users had to change magnification levels to view different types of information within the same context. These inefficiencies represented a barrier

to using the technology even though participants were often able to complete the tasks in the end. In the time-constrained setting of a classroom this could lead to a student falling behind her peers due to the time spent manipulating an ostensibly accessible interface. In other cases “the screen magnifier made it impossible to perform certain actions” such as when “the menu was outside the field of view [and] when she panned to see the menu, she moved the cursor causing the menu to disappear” (p. 7).

When analyzing technical barriers to the accessibility of BBPEs it is helpful to apply the POUR principles underlying the Web Content Accessibility Guidelines (WCAG). POUR stands for Perceivable, Operable, Understandable, and Robust.

- **Perceivable** describes how information is presented to a user. Information is perceivable if the user can access it using one of their senses. Visual information such as pictures can be made perceivable through alternate presentations, such as alt text.
- **Operable** describes how well users can manipulate the interface. For instance, a clickable button can be made operable to users with motor impairments if it can also be activated with the keyboard.
- **Understandable** is the partner to **Perceivable** but includes relationships between pieces of information and controls. For example, a page would be hard to understand if paragraphs were read out in a random order. Setting a fixed navigation order can make a page understandable.
- **Robust** requires that content be well-formed, such that it can be used with a wide range of browsers and assistive technologies. For instance, a web page that only

renders correctly in Edge is less robust than a page that renders correctly in all major browsers.

Code navigation and comprehension are challenging for blind programmers at any age. Under a POUR analysis, screen readers make code perceivable but not understandable: fundamentally, “screen readers produce audio interpretations of user interfaces” (G. Lock et al., 2025, p. 3), but reading the text of a program directly presents issues with punctuation, indentation, and distinguishing between values. As a result “sighted programmers employ a host of different reading strategies at different phases in their work” (p. 6) such as skimming, reading linearly, and debugging. Although “all of these modes are supported in various ways in the visual interfaces of modern editors [...] common screen readers struggle to support these different reading modes, instead providing a simplistic all-or-nothing linearisation of the text” (p. 6). Experienced blind and low vision programmers draw on multiple strategies to navigate code, including using search functionality to find important portions of the code before switching to direct narration. These strategies are not necessarily available to novice programmers, who may not know what to search for, or novice AT users, who may not know advanced screen reader commands.

G. Lock et al. (2025) proposed a shift to navigation based on the program’s abstract syntax tree (AST), saying that “the AST should be treated as the canonical representation [...] and, crucially, the audio presentation should be derived directly from the AST” (p. 6). An AST is an alternative representation of a program that is often used behind the scenes in integrated development environments. When the text of a program is derived from its AST punctuation and indentation are used as visual shorthand to express the structure of the program. It is possible to

derive an equivalent audio representation that uses audio features (e.g. sounds or words) to express the program structure. To facilitate “collaboration amongst peers, and between pupil and teacher,” they argued that the audio representation of code “need not map 100% to the visual presentation” but that the two presentations should be “equivalent, for example sharing referents (such as line numbers, or similar)” (p. 6). These suggestions apply as much to block-based editing as to frame-based editing.

Van der Meulen et al. (2022) interviewed teachers, teaching assistants, and IT experts at schools for children with visual impairment in the Netherlands serving students from kindergarten to the end of high school. The participants were asked about their perception of the usability and accessibility of programming materials for children with visual impairments. The researchers defined accessibility as compliance with the POUR principles and defined usability as “the extent to which a product or service is easily and intuitively understood and navigated through, rooted in the operational experience of the user” (p. 3). These educators expressed that there were multiple accessible outputs from block-based programming environments because of the availability of audible and tangible outputs. This does not mean that all existing BBPEs have accessible outputs; rather, there is a sufficient diversity of BBPE output modalities that the educators were confident in their ability to find an output that was compatible with the needs of their students.

These educators were less sanguine about the accessibility of inputs to block-based programming. They distinguished between low vision and blind children in this arena, pointing to issues with perceivability and operability for blind children. However, they expressed that block-based programming can be accessible to students with low vision, such as with on-screen

magnifiers. Technical issues with development environments persist into university: Baker, Bennett, and Ladner (2019) reported that one of their survey participants avoided learning new integrated development environments (IDEs) during college because of past experiences dedicating hours to learning an environment before finding insurmountable accessibility roadblocks that required them to find a new tool anyway.

Tested Strategies

The previous section discussed a variety of barriers to the accessibility of BBPEs for people with visual impairments. Some researchers have tested remediation strategies with users and reported on the impacts. These user tests are difficult to compare directly due to differences in their participant demographics and methodology. The wide range of strategies tested points to a lack of consensus in the field on how best to address these barriers. Together these studies paint a picture of the strengths and weaknesses of each approach.

Alternatives to Drag and Drop

Milne and Ladner (2018) proposed making block-based programming accessible through touchscreen devices by making the drag part of drag-and-drop programming easier to interact with. The authors conducted a formative study with five children with visual impairments ranging from five to ten years old to test two approaches to moving blocks. The first approach simply added audio feedback to a traditional drag-and-drop interface. The second approach inverted the standard approach to dragging: the user selected a target location in the workspace, which triggered a full-screen menu containing all blocks that could be connected at that location. Users then selected a block from the menu, causing it to be placed at the selected location. The researchers found that “children were not able to use VoiceOver to perform an audio-guided drag

and drop” (p. 8), pointing to the need for alternatives to drag and drop for both low vision and blind users. Children were able to use the “location-first select, select, drop” method, but “liked the idea of selecting a block first and then a location (as in the drag and drop interface) better” (p. 7). They also found that children with low vision “held the iPads close to their faces, making it difficult to both drag the block and see what was going on” (p. 8). After testing the researchers created a hybrid method for moving blocks where users first picked a block in the toolbox and then navigated to select a target location in the workspace. Milne and Ladner concluded that dragging could be replaced with a sequence of selection actions but that two pieces of information had to be communicated non-visually: which mode the application was in and what item was selected.

This study was important because the participants were in elementary school, matching the expected age for beginning users of educational BBPEs. It demonstrated the feasibility of alternative approaches to dragging and dropping that could be used by the target age group. This study also introduced the concept of location-first editing, wherein the user selects the destination before choosing a block to insert. Location-first editing avoids the complexity of previewing a drag to the user through audio; the disadvantage is that the user needs to understand the effects of the insertion before they begin the action. Mountapmbeme and Ludi explored this alternative interaction mode further in a 2025 study.

Tactile interfaces

Tactile interfaces such as tiles and physical blocks are often used to bridge the gap from “unplugged” (screen-free) programming experiences and computer-based programming. StoryBlocks (Koushik et al., 2019) is an example of a tactile programming experience that uses

3d-printed tangible blocks to create audio stories. StoryBlocks was tested with 16 participants aged 11 to 65, including both visually impaired students and staff members who regularly worked with blind students. The researchers wanted to make a programming experience that was equally accessible for blind and sighted users but found that this presented novel design challenges. They found that there was a clear trade-off between the expressivity of their programming language and how simple and easy it was to use. While some of the challenges were specific to tactile blocks, others generalize to screen reader interfaces: “while it is easy to simultaneously compare multiple on-screen objects visually, comparing tactile landmarks is limited by the user’s number of free hands” (p. 9). Comparing two items with a screen reader faces similar issues, since it cannot access both items in parallel. This is not just a technical constraint: if it is easier and more efficient to use the programming blocks visually, friction could arise between blind and sighted partners when trying to work together. Similarly, they found that participants were not always able to identify blocks through touch alone. For sighted users, the meaning of a block is conveyed through multiple types of information such as shape, color, and block text. This study points to the difficulty of identifying blocks to users who are unable to access information in parallel, both when identifying a block in isolation and when comparing multiple blocks.

Figure 3. Blocks connected with notches

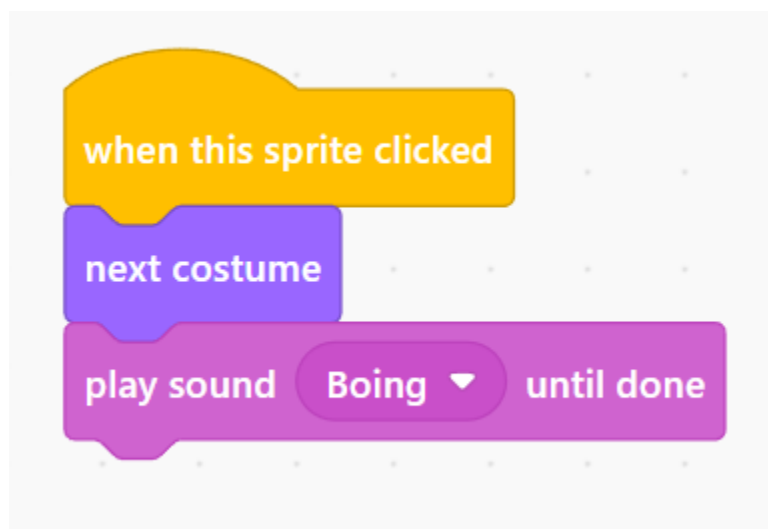


Figure 3. The blocks in this program use vertical notches and tabs to connect in a vertical stack.

StoryBlocks used common block programming features to communicate block type, such as notches and tabs to represent valid connections between blocks. This study is evidence that the basic features of block programming are useful and engaging for blind and low vision students. Despite issues with block identification and comparison “participants spoke positively about the benefits of tangible blocks for diverse learners”, with one teacher saying that “the storytelling approach helped her to understand programming” (Koushik et al., 2019). The blocks provided a useful and fun way to engage with computational thinking skills for blind and low vision students.

Wang and Wagner (2019) took a different approach with their CodeBox64 project by proposing a mouse-based approach to block programming for users with visual impairments. They created an input device to simulate mouse movement with external buttons and provided audio feedback when a block was under the cursor. The input device simulated a joystick with two buttons. The authors tested the usability of CodeBox64 with a group of ten college students

to verify that the device was usable for people without experience programming. Participants used CodeBox64 to write three small programs in the Scratch programming environment without being able to see the screen. The authors concluded that their novel peripheral device could be used to control Scratch, and that their auditory cues about the blocks under the pointer were sufficient for users to complete simple programming tasks. The authors proposed future studies to understand the application of CodeBox64 with more complex programs but have not published any new results.

Quorum Blocks

Figure 4. The Quorum Blocks editor

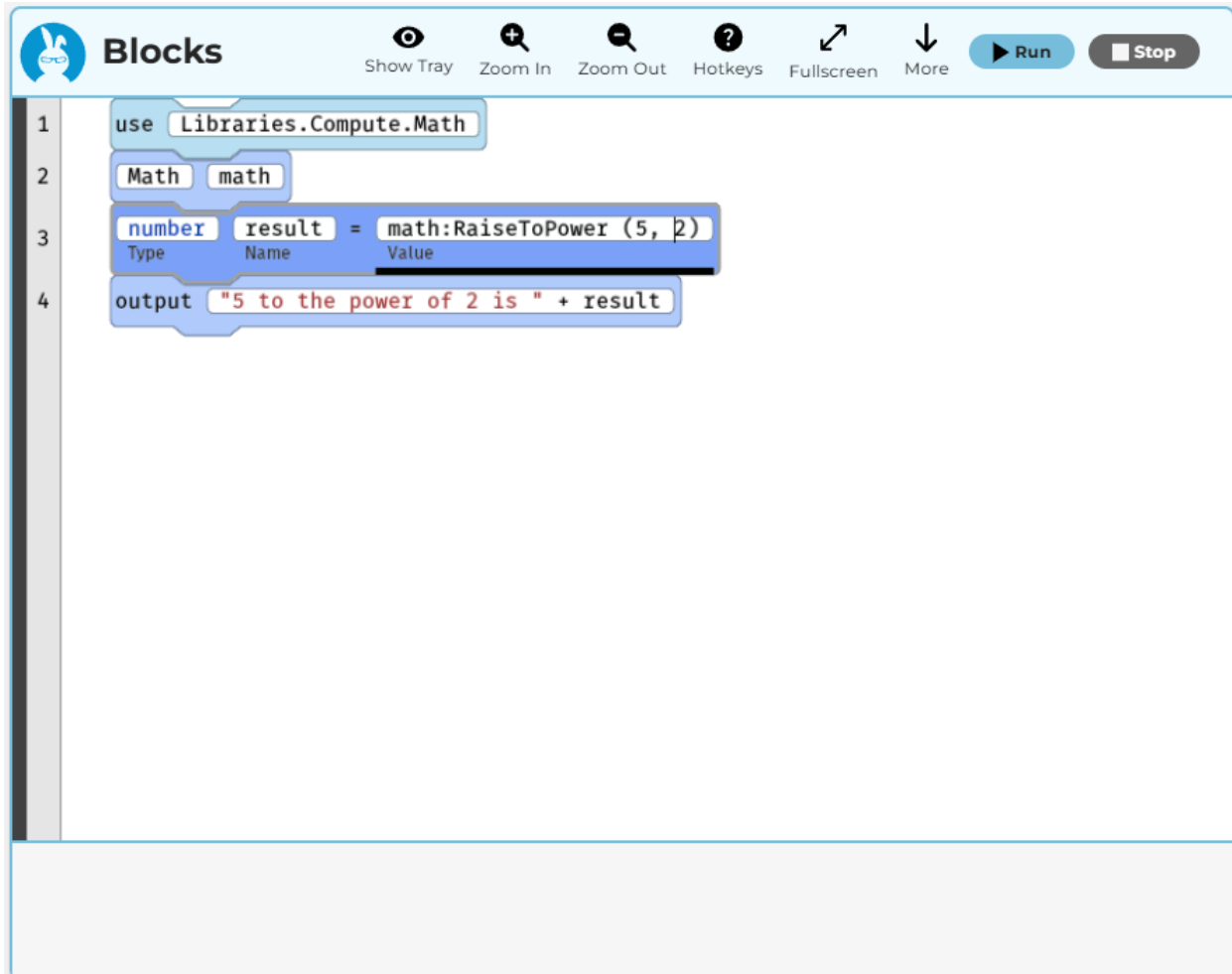


Figure 3. A sample program in the Quorum Blocks editor. The selected block sets the variable named result to "math:RaiseToPower (5, 2)".

Quorum Blocks is a block-based editor interface for the Quorum programming language and environment. The target audience at its introduction in 2024 was “students with disabilities at any time from high school to professional practice” (Stefik et al., p. 5). The goal was for

students with vision to be able to use standard block-based programming actions like drag-and-drop, while students using the keyboard experienced it as a powerful text editor. The interface allowed for a mix of block and text programming, building on prior work by Baker, Milne, and Ladner (2015) and Mountapmbeme, Okafor, and Ludi (2022a). This simplified the experience of writing strings and equations but may create issues with syntax errors. Quorum Blocks provided a wide range of advanced features which were hidden by default but discoverable by students as they learned, including navigation features like “go to line” and “find”. To simplify ordering when traversing through all blocks in a program, they rendered blocks in a single column from top to bottom, eschewing the two-dimensional workspaces of more traditional BBPEs. Quorum Blocks was evaluated through three focus groups with adults and high school students with disabilities, who completed a data science lesson and gave formative feedback on the environment. More work is needed to understand the implications of each of these design choices, but Quorum Blocks represents an interesting crossover point between block code and text code that is in active use today.

Navigation

StructJumper was a 2015 project from Baker, Milne, and Ladner that “create[d] a hierarchical tree of the nesting structure of a program [...] to allow users to both navigate within the program and gain an understanding of the structure of code within the program” (p. 1). This hierarchical structure is similar to an abstract syntax tree. The goal of the tool was to help users “switch between being able to make edits within the code and gaining contextual information without losing her place” (p. 2). The target audience for StructJumper was blind developers, so StructJumper was created as a plugin for the Eclipse Integrated Development Environment

(IDE). The tool was tested with seven blind programmers, with an average participant age of 24.1 and an average of 7.8 years of programming experience. Participants completed tasks with and without the StructJumper plugin and results were measured with task completion time, task correctness, and participant experience questions. The first two tasks tested navigation by requiring participants to find locations in code where specific conditions were true. The third task asked participants to describe the conditions necessary to reach a particular line of code, which required them to navigate through multiple levels of context. Participants reported positive sentiments about the tool, although there was not a statistically significant effect of the tool on either task scores or completion time.

Schanzer et al. (2019) performed a similar analysis of AST-based navigation in their CodeMirror-Blocks environment. Their system renders code as collapsible blocks, which is semantically equivalent to the tree structure used in StructJumper. CodeMirror-Blocks included a labeling layer that allowed for natural-language descriptions of nodes, separating the audio representation of a node from its textual representation. The researchers evaluated CodeMirror-Blocks with thirteen blind programmers who completed three navigation and comprehension tasks. The researchers controlled for the effects of variable experience with the language syntax by using a programming language that was not familiar to any of the participants. They found that participants had an easier time orienting themselves in code when using CodeMirror-Blocks, appreciated the structural navigation and removal of syntax barriers, and “indicated that things ‘felt faster’ when using CMB” (p. 6).

Takamatsu et al. (2025) proposed “a system that utilizes an LLM [Large Language Model] to automatically segment source code into semantically coherent sections and generate

explanatory descriptions for each section” (p. 1). Their system analyzes block-based code and uses an LLM to first break the code into pieces and then describe each segment of code in natural language. The resulting information is presented by rewriting the block code into annotated sections that can be expanded and explored. Their system attempts to address barriers to navigation and comprehension of code, particularly for “developing a coherent understanding of program structure and logic” (p. 11). The authors tested their system with twenty participants between the ages of 21 and 24, who completed programming tasks with the LLM-based interface and with a conventional block programming interface. Participant demographics limit the generalizability of this study: most of the participants were experienced programmers and none of them were visually impaired. Participants wore blindfolds while completing the study tasks.

The coding tasks focused on navigation and comprehension of code and were similar to the tasks used to test StructJumper (Baker, Milne, and Ladner, 2015). The first task required participants to “locate a specific line of code that outputs the value of a designated variable” (Takamatsu et al., 2025, p. 12). The second task asked the students to “examine its surrounding logic [...] and explain, in their own words, under what conditions the variable would be printed” (p. 12). In the last task, the participants had to explain what values would be produced by a print statement, and under what conditions. This final task tested participants’ ability to put together information from multiple places in the program to trace program execution through loops and conditionals.

The researchers looked at four types of data to assess the effectiveness of their LLM-based interface: task accuracy, task completion time, subjective evaluation, and open-ended feedback. A key finding was that “segmentation effectively reduces the mental load imposed by

unstructured code” and “clearly defined segment boundaries and concise section summaries were highlighted as valuable features that support both navigation and conceptual understanding” (Takamatsu et al., 2025, p. 16). The authors found interesting limitations to the effectiveness of their segmented interface because users varied in their preferences for how segment descriptions should be presented. The authors concluded that users would need to be able to customize the output to their own preferences. This potential solution also suffered from standard caveats about trusting the output of LLMs, which still struggle with reproducibility and hallucinations.

Taken together, these studies show that navigation based on an abstract syntax tree can be an effective way to reduce cognitive load when reading code. Although all three studies tested with adult programmers, reducing cognitive load is valuable in education contexts: it expands the complexity and interest level of programs that beginners can be exposed to before reaching an overloaded state. The AST-based navigation strategies proposed in this section can be adapted to block-based programming environments, which use an AST as the base representation of code.

Blockly modifications

Three papers proposed and tested customizations of Blockly-based environments to improve accessibility for users with visual impairments. Tabassum et al. (2024) created a Blockly-based environment called GridWorld that is designed for young learners as they are introduced to programming. Students used blocks to program the behavior of sprites on a grid, which is a common activity in introductory programming classes. The researchers provided audio feedback through the JavaScript Speech Synthesis API. The feedback could be triggered by hovering over contents of the blocks workspace or the gride environment, or through keyboard actions. They also implemented “a range of tones to acknowledge the student’s actions

and progress in different contextual scenarios” (p. 4) that represented an interesting mixture of speech and non-speech feedback.

GridWorld provided 25 different keyboard shortcuts for navigating and editing the program and the execution environment (Tabassum et al., 2024). It included a debugger so that students could run code one block at a time, then pause to explore the state of the execution environment. The simple shortcuts and debugging experience were valuable affordances given the target audience of middle schoolers. The use of the Speech Synthesis API allowed them to provide contextual information in speech output without dealing with the technical complexity required to communicate with a screen reader. However, this approach does not meet the robustness requirement of POUR because it overrides screen reader customizations chosen by the user.

The researchers evaluated GridWorld through a pilot study with 19 sighted middle school students and an experimental session with five visually impaired middle and high school students (Tabassum et al., 2024). The sighted students were able to use GridWorld to complete five programming activities. Their feedback was used to fix bugs and improve the program before testing with visually impaired students. Five visually impaired students participated in a facilitated training session on using GridWorld, but only one went on to complete evaluation tasks at home. This limits the generalizability of the GridWorld study for visually impaired students; the primary finding was that sighted students were able to use the system, including keyboard shortcuts.

Two papers out of Stephanie Ludi’s lab modified the Blockly library with a goal of improving navigation tasks for visually impaired programmers. Mountapmbeme, Okafor, and

Ludi (2022b) presented a prototype of their Accessible Blockly library, which built screen reader and keyboard navigation support on top of the Blockly library. They tested a “spatial navigation” approach using left/right and up/down navigation to move between and within blocks. Their implementation “defined a virtual cursor on the programming environment controlled by keystrokes” (p. 4) with two modes: navigation and editing. In both modes the cursor could be moved using the WASD keys as arrow keys, as is common in gaming. Navigation mode could be used to explore code by moving between blocks on the workspace. Edit mode could be used to add blocks to existing programs or build entirely new programs with a “location-first select, then block select method” (p. 5) for block insertion. The second module of Accessible Blockly added speech output, including descriptions of blocks that could add contextual information and deviate from the display text of the blocks.

The researchers evaluated their system through a navigation-only study with 12 blind programmers. The researchers noted that editing is also important, but that “accessibility of code navigation is commonly assessed in isolation” and “navigation is crucial in writing code” (Mountapmbeme et al., 2022b, p. 6). Ten out of twelve of the participants reported that they used screen readers daily. The ages ranged from 22 to 60, with an average age of 41. The tasks used for evaluation were similar to those used for StructJumper and other navigation-focused interventions. During evaluation of Accessible Blockly, the control environment used a default cursor that combined edit mode and navigation mode. This was the keyboard navigation system built into the Blockly library at the time.

Participants found Accessible Blockly easier to use than the default cursor, but that both navigation schemes were accessible and usable (Mountapmbeme et al., 2022b). Task completion

time was faster with Accessible Blockly than with the default cursor. *Users thought that Accessible Blockly was less cognitively demanding because it made it easier to get an overview of the program. This decrease in cognitive load could be important for beginning programmers.* However, participants with more programming experience thought that the default cursor was more effective for navigating complex programs. Feedback on the two systems revealed a tension between simplicity for novice users and completeness for advanced users and complex systems. Study participants also gave suggestions for how to improve the accessibility of block-based code more generally, which focused on improving the screen reader output to have more or different information about the individual blocks.

A second study in 2025 evaluated the usability of the Accessible Blockly system for editing code (Mountapmbeme & Ludi, 2025). The researchers evaluated the system with eleven blind adults, including eight who participated in the previous study. The evaluation consisted of a training period (approximately 25 minutes) followed by the completion of three tasks: 1) write a simple program with a loop, 2) edit a program by finding and deleting a block, and 3) modify the logic of an existing program to change its output under certain conditions. These three tasks exercised a variety of common actions, such as finding blocks in the toolbox and workspace and deleting blocks. Participants were generally able to complete the tasks in the allotted time. They also rated how difficult it was to complete five types of actions: “locating an insertion point on the workspace, selecting a block from the toolbox, moving a block from the toolbox to the workspace, deleting a block on the workspace, and determining one’s location within code on the workspace” (p. 22). Participants gave high scores to all actions except for awareness of their

location within the code. The authors noted that bugs in the program may have contributed to this difficulty.

Feedback on Accessible Blockly was positive, with participants finding it usable for both constructing and editing block-based programs. They said that it was easy to use, and that it had a “shallow learning curve for new users” (Mountapmbeme & Ludi, 2025, p. 23). The positive feedback demonstrated that speech output could effectively support reading, writing, and editing block-based code. A limitation of the study is the demographics: all participants were adults and had significant experience with both screen readers and keyboards. Nevertheless, these results are promising evidence for the effectiveness of keyboard navigation and screen reader support in BBPEs.

Mollik et al. (2026) built on prior work by building keyboard shortcuts and screen reader support as a Blockly plugin that could be used “with any Blockly-based platform” (p. 7). The system consisted of a navigation module, an audio module, and an accessibility layer. The navigation model extended prior work by adding an additional dimension for moving in and out of loops and other nested code. The audio module “generate[d] voice feedback and auditory cues” (p. 3) in a standards-compliant way, while the accessibility layer added supporting features such as labeling and jumping to stacks. The researchers evaluated their Extension Accessibility Framework (EAF) approach by using it in two existing Blockly-based environments “without having to modify the core library”, meaning that “existing BBPEs such as Scratch, MakeCode, and Code.org can potentially adopt EAF without restructuring their current implementations, lowering the barrier to widespread accessibility adoption” (p. 7). The authors built a test suite containing 177 test cases across a variety of navigational and editing scenarios. These tests

represent a substantial contribution to the field, providing a template for testing other systems.

They also tested the system with VoiceOver, JAWS, and NVDA and conducted task-based evaluations of the interface with four adult participants, who were successful in using it to complete tasks. An interesting finding about the navigational model was the “undergraduate participants found the shortcuts ‘easy to remember’ while expert participants found them ‘tied to program scope’, show[ing] that the navigation model works for users with different levels of expertise” (p. 7).

Guidelines

Several authors wrote guidelines for creating accessible instructional technologies based on their research. Some of the guidelines were general-purpose, while others were specific to programming or even to block programming. The common features of these guidelines represent consensus in the literature about the goals and features of accessible instructional technology, including accessible BBPEs.

The first common feature relates to the robustness axis of POUR. Shaheen (2024) said that accessible BBPEs should meet WCAG standards and work on multiple operating systems, while Gadiraju et al. (2021) emphasized the importance of designing for existing technology. Mountapmbeme and Ludi (2025) said that technical solutions must “be easy to integrate into existing BBPEs” and “provide equivalent access to all programming features” (p. 4). For all accessible software it is important to “avoid glitches” (Mountapmbeme & Ludi, 2025, p. 27) because users may be unable to access alternate forms of information to work around bugs. It is necessary to build robust, well-tested systems that work on all operating systems and integrate

with existing technology such as screen readers and refreshable Braille displays. Failures of robustness have consequences ranging from social exclusion to outright inaccessibility.

Second, accessible BBPEs must address issues of social barriers and communication in the classroom. To do this, they should “facilitate communication between BLV and sighted users” (Shaheen, 2024, p. 24), support “independent play and interdependent collaboration (Gadiraju et al., 2021, p. 10), and be “usable by sighted students and visually impaired students, including in collaboration” (Mountapmbeme et al., 2022a, p. 4). Communication between teachers and students is equally important, and extends to classroom resources and curriculum. Accessible BBPEs should “help teachers produce accessible spatial information” (Shaheen, 2024, p. 24), “use familiar and consistent terminology” (Mountapmbeme & Ludi, 2025, p. 26), and “design for an unpredictable mix of classroom and remote learning” (Gadiraju et al., 2021, p. 11).

Third, accessible BBPEs must be easy to learn and simple to use because their users may not have pre-existing computer or AT skills. To address this, they should “provide scaffolding for novice AT users” and “create simple interfaces” (Shaheen, 2024, pp. 23-24). Systems should also “design to maximize knowledge transfer” (Mountapmbeme & Ludi, 2025, p. 26), which means using conventional shortcuts and interaction patterns wherever possible. Since users will grow in their skills over time, accessible BBPEs must provide help pages and user manuals that can be accessed as the need arises. Because user needs vary widely, systems should “design for feedback at different levels of verbosity” and “design to include search functionality” (Mountapmbeme & Ludi, 2025 pp. 25-26).

Chapter 3

Conclusion

The accessibility of block-based programming environments emerged as a research topic in the past fifteen years. Given the prevalence of BBPEs in early computing education, improving the accessibility of BBPEs is a critical task for the field in the next five years. Alternatives such as tactile and textual programming environments are not a drop-in replacement for BBPEs. The educational affordances of block-based programming environments include application-specific blocks, and protection against syntax errors. These advantages are useful for blind and low vision students, even if they cannot use the spatial metaphors and drag-and-drop interfaces of BBPEs. Accessible BBPEs are important for social inclusion of BLV students in classrooms with sighted peers and can be designed to be usable by both visually impaired and sighted students.

The existing literature demonstrates several ways that existing BBPEs are not accessible to blind and low vision students, ranging from technical affordances to social and contextual factors in the classroom. There is no consensus within the literature on a single correct way to bring educational BBPEs into alignment with technical or regulatory standards. However, there is consensus on guidelines and minimum requirements that center the needs of students in the classroom.

Opportunities for Future Work

Based on this literature, I propose future work in creating standardized keyboard interfaces for BBPEs at a library level: the Blockly library should implement a cursor-based

keyboard interface to meet accessibility standards and address the needs of learners with visual and motor impairments. In the following sections I justify this conclusion, drawing from my knowledge of the landscape of educational BBPEs and my understanding of the technical constraints inherent in these environments.

Cursors

Computer interfaces often expose a cursor that indicates user focus and the target location for keyboard interactions. There are strong UX conventions around the use of a cursor: the arrow keys move the cursor in a logical navigation order; the user may click to move the cursor to a location indicated by the mouse pointer; insertion and deletion occur at the cursor. Screen readers apply the concept of a cursor to all user interfaces, allowing users to traverse web pages as though they were text documents. Users who are familiar with laptops and desktops will find a cursor intuitive; novices who learn about the concept from BBPEs will be able to transfer their knowledge to other environments.

Addressing Social Barriers

Multiple researchers found that blind and visually impaired individuals are often reluctant to use assistive technology for social reasons. Not wanting to be visibly different emerged as a key barrier. Students in non-specialist educational environments did not have sufficient support in learning how to use their assistive technology, and mainstream teachers rarely knew how to help. In some cases using AT created a barrier to side-by-side collaboration because a sighted person did not understand how to interact with it.

Building accessibility features into mainstream products and experiences, such as by adding keyboard navigation as a core feature in all BBPEs, is a way to address some of these

social barriers. A cursor-based workflow provides a clear bridge between keyboard and mouse control of an editor, facilitating pair programming with sighted peers and debugging with teacher help. Using a simple set of shortcuts that mirror standard cursor navigation in a text document makes it easy for non-specialist teachers who may be unfamiliar with screen readers to learn keyboard controls. They can then help students directly rather than requiring the intervention of a TSVI.

Keyboard controls improve accessibility for a range of students including those with motor impairments and those with poor or broken touchpads. Some sighted students may prefer to use the keyboard to edit code quickly or to track their position in code as they read. Supporting switching between the keyboard and mouse means that BLV students can pair program with sighted students or ask their sighted teachers to check their work and help with debugging. In this sense, keyboard controls act as a curb cut that improves usability for a larger audience than the original intent.

Addressing Age-Related Barriers

Implementing custom keyboard controls that are independent of screen reader norms always requires justification. Learning a new set of custom keyboard controls places a cognitive burden on the user and is generally discouraged. However, in this context it is justified to create a system of keyboard controls based on the arrow keys and single-letter shortcuts. In this context the decision is not between known screen reader controls and unknown custom controls; rather, the decision is which set of controls to learn.

Educational BBPEs are often used in elementary and middle school. According to Shaheen (2024), that is before many BLV students begin to learn how to use screen readers.

Since BBPEs are used with such very young learners, any solutions to accessibility issues must not rely on students already knowing how to use screen readers, refreshable Braille displays, or other AT. Knowing how to use a screen reader should not be a barrier to block-based programming.

In a similar vein, solutions to accessibility issues should not rely on advanced keyboarding skills from beginner users. Shortcuts that require the user to press and hold multiple keys (chorded shortcuts) may present obstacles to young learners. Basic actions should rely on single-key shortcuts, while advanced actions can use chorded shortcuts.

Addressing Technical Barriers

Any effort to improve the accessibility of BBPEs must include work at a library level to provide a consistent interface to the maximum number of users. Gadiraju et al. (2021) said that instructional tools for blind students should apply existing technology and focus on discrete tools. Mollik et al. (2026) approached this in their paper on an Extension-Based Accessibility Framework for Blockly, where they created an accessibility plugin that is compatible with the Blockly library and validated its use in multiple Blockly-base applications. Many educational BBPEs are built on top of the Blockly library. Each application exposes a unique set of blocks and fields to the users, but the shared library handles the technical implementation of block rendering and editing.

Implementing accessibility features in the Blockly library is a logical extension of this work that will ensure consistency across a range of BBPEs, centralize development and testing efforts, and guarantee that all standard user actions are made accessible. Blockly-level changes can easily be propagated to a large number of deployed BBPEs. Each application will also need

to ensure the accessibility of all other portions of their programming environments to ensure compliance with WCAG. Application-level user education will also be critical.

Meeting accessibility standards

Implementing a cursor has technical benefits for screen reader compatibility and WCAG support. The Blockly library can provide meaningful textual descriptions of cursor locations, which can be exposed through a screen reader or a refreshable braille display. Cursor movement can be described spatially and semantically, addressing the perceivable and understandable portions of POUR. Maintaining a cursor also addresses the operable axis by clearly indicating where and how keyboard controls will interact with the page. Finally, the system would be robust by working across a wide range of hardware and software platforms.

References

- Baker, C. M., Bennett, C. L., & Ladner, R. E. (2019). Educational Experiences of Blind Programmers. *Proceedings of the 50th ACM Technical Symposium on Computer Science Education, SIGCSE '19*, 759–765. <https://doi.org/10.1145/3287324.3287410>
- Baker, C. M., Milne, L. R., & Ladner, R. E. (2015). StructJumper: A Tool to Help Blind Programmers Navigate and Understand the Structure of Code. *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems, CHI '15*, 3043–3052. <https://doi.org/10.1145/2702123.2702589>
- Baker, C. M., Milne, L. R., & Ladner, R. E. (2019). Understanding the Impact of TVIs on Technology Use and Selection by Children with Visual Impairments. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, CHI '19*, 1–13. <https://doi.org/10.1145/3290605.3300654>
- Fact Sheet: New Rule on the Accessibility of Web Content and Mobile Apps Provided by State and Local Governments.* (2024, March 8). ADA.Gov. <https://www.ada.gov/resources/2024-03-08-web-rule/>
- G. Lock, J., C. C. Brown, N., & Kölling, M. (2025). Programming Education for Blind and Low Vision Users: Beyond Reading the Screen. *Proceedings of the 25th Koli Calling International Conference on Computing Education Research, Koli Calling '25*, 1–7. <https://doi.org/10.1145/3769994.3770018>

- Gadiraju, V., Doyle, O., & Kane, S. (2021). Exploring Technology Design for Students with Vision Impairment in the Classroom and Remotely. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems, CHI '21*, 1–13. <https://doi.org/10.1145/3411764.3445755>
- Google for Developers. (2024). *Blockly*. Google for Developers. <https://developers.google.com/blockly>
- Huff, E. W., Boateng, K., Moster, M., Rodeghero, P., & Brinkley, J. (2021). Exploring the Perspectives of Teachers of the Visually Impaired Regarding Accessible K12 Computing Education. *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education, SIGCSE '21*, 156–162. <https://doi.org/10.1145/3408877.3432418>
- Individuals with Disabilities Education Act, 20 U.S.C. 1401 § 602 (2004). <https://www.federalregister.gov/d/06-6656/p-3996>
- Introduction to the Americans with Disabilities Act*. (2026, March 3). ADA.Gov. <https://www.ada.gov/topics/intro-to-ada/>
- Koushik, V., Guinness, D., & Kane, S. K. (2019). StoryBlocks: A Tangible Programming Game To Create Accessible Audio Stories. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, CHI '19*, 1–12. <https://doi.org/10.1145/3290605.3300722>
- Milne, L. R., & Ladner, R. E. (2018). Blocks4All: Overcoming Accessibility Barriers to Blocks Programming for Children with Visual Impairments. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems, CHI '18*, 1–10. <https://doi.org/10.1145/3173574.3173643>
- Mollik, R. H., Kosuri, V. K., Djalali, H., Ludi, S., & Mountapmbeme, A. (2026). *An Extension-Based Accessibility Framework for Making Blockly Accessible to Blind and Low-Vision Users* (arXiv:2601.10688; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2601.10688>

- Mountapmbeme, A., & Ludi, S. (2020). Investigating Challenges Faced by Learners with Visual Impairments using Block-Based Programming/Hybrid Environments. *Proceedings of the 22nd International ACM SIGACCESS Conference on Computers and Accessibility, ASSETS '20*, 1–4. <https://doi.org/10.1145/3373625.3417998>
- Mountapmbeme, A., & Ludi, S. (2021). How Teachers of the Visually Impaired Compensate with the Absence of Accessible Block-Based Languages. *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility, ASSETS '21*, 1–10. <https://doi.org/10.1145/3441852.3471221>
- Mountapmbeme, A., & Ludi, S. (2025). Designing accessible block-based programming environments for persons with visual impairments. *Universal Access in the Information Society*, 24(3), 2351–2382. <https://doi.org/10.1007/s10209-025-01200-y>
- Mountapmbeme, A., Okafor, O., & Ludi, S. (2022a). Accessible Blockly: An Accessible Block-Based Programming Library for People with Visual Impairments. *Proceedings of the 24th International ACM SIGACCESS Conference on Computers and Accessibility, ASSETS '22*, 1–15. <https://doi.org/10.1145/3517428.3544806>
- Mountapmbeme, A., Okafor, O., & Ludi, S. (2022b). Addressing Accessibility Barriers in Programming for People with Visual Impairments: A Literature Review. *ACM Trans. Access. Comput.*, 15(1), 7:1-7:26. <https://doi.org/10.1145/3507469>
- Schanzer, E., Bahram, S., & Krishnamurthi, S. (2019). Accessible AST-Based Programming for Visually-Impaired Programmers. *Proceedings of the 50th ACM Technical Symposium on Computer Science Education, SIGCSE '19*, 773–779. <https://doi.org/10.1145/3287324.3287499>

- Shaheen, N. L. (2024). Exploring Blind and Low-Vision Youth's Digital Access Needs in School: Toward Accessible Instructional Technologies. *ACM Trans. Access. Comput.*, 17(3), 15:1-15:31. <https://doi.org/10.1145/3688805>
- Statistics on Children*. (2026, March). The American Foundation for the Blind. <https://www.afb.org/research-and-initiatives/statistics/children-youth-vision-loss>
- Stefik, A., Allee, W., Contreras, G., Kluthe, T., Hoffman, A., Blaser, B., & Ladner, R. (2024). Accessible to Whom? Bringing Accessibility to Blocks. *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1, SIGCSE 2024*, 1286–1292. <https://doi.org/10.1145/3626252.3630770>
- Szpiro, S. F. A., Hashash, S., Zhao, Y., & Azenkot, S. (2016). How People with Low Vision Access Computing Devices: Understanding Challenges and Opportunities. *Proceedings of the 18th International ACM SIGACCESS Conference on Computers and Accessibility, ASSETS '16*, 171–180. <https://doi.org/10.1145/2982142.2982168>
- Tabassum, M., Gray, J., & Smith, D. (2024). An Accessible Blocks Language for Students with and without Visual Impairments. *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1, SIGCSE 2024*, 1300–1306. <https://doi.org/10.1145/3626252.3630775>
- Takamatsu, N., Kamioka, E., Minh Tran, C., & Tan, P. X. (2025). Enhancing Algorithm Comprehension for Visually Impaired Individuals With LLM-Based Programming Code Segmentation. *IEEE Access*, 13, 204040–204057. <https://doi.org/10.1109/ACCESS.2025.3638804>
- van der Meulen, A., Hartendorp, M., Voorn, W., & Hermans, F. (2022). The Perception of Teachers on Usability and Accessibility of Programming Materials for Children with Visual Impairments. *ACM Trans. Comput. Educ.*, 23(1), 14:1-14:21. <https://doi.org/10.1145/3561391>

Wang, Z., & Wagner, A. (2019). Evaluating a Tactile Approach to Programming Scratch. *Proceedings of the 2019 ACM Southeast Conference, ACMSE '19*, 226–229.

<https://doi.org/10.1145/3299815.3314464>

Weintrop, D., & Wilensky, U. (2017). Comparing Block-Based and Text-Based Programming in High School Computer Science Classrooms. *ACM Transactions on Computing Education*, 18(1), 1–25. <https://doi.org/10.1145/3089799>

Yongpradit, P., Weissman, H., & Ellis, V. (2025). *2025 State of AI + CS Education*. Code.org.

<https://drive.google.com/file/d/1p1x-UFVvec0fwXOBv2B17j65wiD8KYSvQ/view>